SQL PROJECT

SQL Query Practice & Problem Solving

PROJECT REPORT

Prepared by: Raja Kumar

Project focus	SQL query practice and problem solving
Learning progression	Fundamentals → aggregation and joins → advanced analytical SQL
Practice platforms	W3Schools, SQL-Practice, HackerRank, DataLemur, and SQLZoo
Document contents	Project report, representative SQL solutions, explanations, and best practices

HackerRank Badge: - Raja Kumar - rajakumarintech | HackerRank

This report is based on the SQL roadmap supplied. It distinguishes representative solutions from questions that require their full original schema and prompt for an exact answer.

1. Project Overview

This project is a structured SQL practice and problem-solving portfolio. The roadmap organises learning across W3Schools, SQL-Practice, HackerRank, DataLemur, and SQLZoo. It starts with core SQL syntax and progresses to aggregation, joins, and more challenging interview-style problems.

The objective is to build practical confidence in writing readable queries, retrieving relevant data, summarising records, connecting related tables, and solving analytical questions. The supplied roadmap is a checklist of topics and exercise names, rather than the full text and schema for every external problem. For that reason, this report includes representative solutions and clearly identifies where an exact platform answer depends on additional details.

2. Project Objectives

- Build a strong foundation in SQL syntax and relational data retrieval.
- Practise filtering, sorting, grouping, aggregation, and joins.
- Understand NULL handling, string matching, set operations, and aliases.
- Apply SQL to common technical interview and data-analysis problems.
- Develop reusable query patterns for Easy, Medium, and Hard exercises.
- Maintain a professional record of the learning roadmap, solutions, and query logic.

3. Project Roadmap

Level	Platform / Section	Main focus
Level 1	W3Schools	SQL exercises and core SQL categories.
Level 2	SQL-Practice	Basic and advanced SQL practice, including Easy, Medium, and Hard questions.
Level 3	HackerRank	SELECT queries, aggregation, joins, and interview-style SQL.
Level 4	DataLemur	SQL interview questions and applied analytical problems.
Level 5	DataLemur / advanced sections	Advanced and Hard SQL challenges.
Additional practice	SQLZoo	Supplementary SQL problem-solving practice.

4. Core SQL Concepts

Concept	Purpose	Example
SELECT	Retrieves required columns.	SELECT name, salary FROM employees;
WHERE	Filters rows by a condition.	SELECT * FROM employees WHERE salary > 50000;
DISTINCT	Removes duplicate result values.	SELECT DISTINCT department_id FROM employees;
ORDER BY	Sorts the result set.	SELECT * FROM employees ORDER BY salary DESC;
GROUP BY	Groups rows for aggregate calculations.	SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;
HAVING	Filters grouped/aggregated results.	SELECT department_id, COUNT(*) FROM employees GROUP BY department_id HAVING COUNT(*) > 5;
JOIN	Combines related rows from multiple tables.	SELECT e.name, d.department_name FROM employees e JOIN departments d ON e.department_id = d.department_id;
LIKE	Matches text patterns.	SELECT * FROM customers WHERE name LIKE 'A%';
IN / BETWEEN	Tests membership or an inclusive range.	SELECT * FROM employees WHERE department_id IN (1,2,3);
Aggregates	COUNT, AVG, SUM, MIN, and MAX summarise data.	SELECT AVG(salary) FROM employees;
UNION	Combines compatible result sets and removes duplicates.	SELECT city FROM customers UNION SELECT city FROM suppliers;
NULL	Tests missing or unknown values.	SELECT * FROM employees WHERE manager_id IS NULL;

5. Representative Solved SQL Exercises

The examples below demonstrate the SQL patterns named in the roadmap. Table and column names are illustrative unless the problem is explicitly tied to a standard exercise schema.

5.1 SELECT — Retrieve all customers

```
SELECT *
FROM Customers;
```

Explanation: Returns every column and row in Customers.

5.2 WHERE — Customers in Germany

```
SELECT *
FROM Customers
WHERE Country = 'Germany';
```

Explanation: Filters rows to customers whose country is Germany.

5.3 ORDER BY — Sort by country

```
SELECT *
FROM Customers
ORDER BY Country ASC;
```

Explanation: Sorts the output alphabetically by country.

5.4 DISTINCT — Unique countries

```
SELECT DISTINCT Country
FROM Customers;
```

Explanation: Returns each country value once.

5.5 COUNT — Count customers

```
SELECT COUNT(*) AS customer_count
FROM Customers;
```

Explanation: Counts the rows in Customers.

5.6 GROUP BY — Customers per country

```
SELECT Country, COUNT(*) AS customer_count
FROM Customers
GROUP BY Country;
```

Explanation: Returns one row per country with its customer count.

5.7 HAVING — Countries with more than five customers

```
SELECT Country, COUNT(*) AS customer_count
FROM Customers
GROUP BY Country
HAVING COUNT(*) > 5;
```

Explanation: Filters grouped results based on the aggregate count.

5.8 LIKE — Names starting with A

```
SELECT *
FROM Customers
WHERE CustomerName LIKE 'A%';
```

Explanation: The % wildcard matches zero or more characters.

5.9 IN — Customers in selected countries

```
SELECT *
FROM Customers
WHERE Country IN ('Germany', 'France', 'Spain');
```

Explanation: Matches any one of the listed country values.

5.10 BETWEEN — Products priced from 10 to 50

```
SELECT *
FROM Products
WHERE Price BETWEEN 10 AND 50;
```

Explanation: BETWEEN includes both boundary values.

5.11 JOIN — Employee and department names

```
SELECT e.name, d.department_name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id;
```

Explanation: Joins employee records to their related department.

5.12 NULL — Employees without a manager

```
SELECT *
FROM employees
WHERE manager_id IS NULL;
```

Explanation: Uses IS NULL to identify missing manager values.

5.13 UPDATE — Increase prices in one category

```
UPDATE Products
SET Price = Price * 1.10
WHERE CategoryID = 2;
```

Explanation: Raises prices by 10% for rows matching CategoryID 2. Verify the filter before running.

5.14 DELETE — Delete a customer by ID

```
DELETE FROM Customers
WHERE CustomerID = 91;
```

Explanation: Removes the matching customer row. Confirm the ID and backup requirements first.

5.15 UNION — Combine city lists

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers;
```

Explanation: Combines compatible city lists and removes duplicate values.

6. HackerRank-Style Solutions

6.1 Revising the Select Query I

```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'USA'
  AND POPULATION > 100000;
```

Explanation: Returns all CITY columns for USA records with population above 100,000.

6.2 Revising the Select Query II

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'USA'
  AND POPULATION > 120000;
```

Explanation: Returns city names for USA records with population above 120,000.

6.3 Select All

```
SELECT *
FROM CITY;
```

Explanation: Returns all rows and columns from CITY.

6.4 Select By ID

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

Explanation: Returns the CITY record whose ID is 1661.

6.5 Japanese Cities' Attributes

```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

Explanation: Returns all columns for Japanese cities.

6.6 Japanese Cities' Names

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

Explanation: Returns only the names of Japanese cities.

6.7 Weather Observation Station 1

```
SELECT CITY, STATE
FROM STATION;
```

Explanation: Returns the city and state columns.

6.8 Weather Observation Station 3

```
SELECT DISTINCT CITY
FROM STATION
WHERE MOD(ID, 2) = 0;
```

Explanation: Returns unique city names for rows with even IDs. Some engines use ID % 2 = 0.

6.9 Weather Observation Station 4

```
SELECT COUNT(CITY) - COUNT(DISTINCT CITY) AS duplicate_city_count
FROM STATION;
```

Explanation: Calculates the difference between city entries and distinct city names.

6.10 Employee Salaries — Employee earns more than manager

```
SELECT e.name
FROM Employee e
JOIN Employee m
  ON e.managerId = m.id
WHERE e.salary > m.salary;
```

Explanation: Uses a self-join to compare an employee's salary with their manager's salary.

6.11 Second Highest Salary

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Explanation: Returns the second-highest distinct salary, or NULL if it does not exist.

6.12 Top Three Salaries by Department

```
WITH ranked AS (
  SELECT e.name, d.department_name, e.salary,
         DENSE_RANK() OVER (
           PARTITION BY e.department_id
           ORDER BY e.salary DESC
         ) AS salary_rank
  FROM employees e
  JOIN departments d
    ON e.department_id = d.department_id
)
SELECT department_name, name, salary
FROM ranked
WHERE salary_rank <= 3
ORDER BY department_name, salary DESC, name;
```

Explanation: DENSE_RANK ranks salary levels within each department and includes ties.

7. Additional Advanced SQL Patterns

7.1 Customers who never placed an order

```
SELECT c.name AS customer_name
FROM Customers c
LEFT JOIN Orders o ON o.customerId = c.id
WHERE o.id IS NULL;
```

Explanation: The LEFT JOIN retains customers without a matching order; the NULL filter selects those customers.

7.2 Duplicate email addresses

```
SELECT email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

Explanation: Groups email addresses and returns those appearing more than once.

7.3 Department highest salary

```
WITH ranked AS (
  SELECT e.name, d.name AS department, e.salary,
         DENSE_RANK() OVER (
           PARTITION BY e.departmentId
           ORDER BY e.salary DESC
         ) AS salary_rank
  FROM Employee e
  JOIN Department d ON d.id = e.departmentId
)
SELECT department, name, salary
FROM ranked
WHERE salary_rank = 1;
```

Explanation: Returns all employees tied for the highest salary in each department.

7.4 Consecutive numbers

```
SELECT DISTINCT a.num AS ConsecutiveNums
FROM Logs a
JOIN Logs b ON b.id = a.id + 1 AND b.num = a.num
JOIN Logs c ON c.id = b.id + 1 AND c.num = b.num;
```

Explanation: Finds values repeated across three consecutive log IDs.

7.5 Page with no likes

```
SELECT p.page_id
FROM Pages p
LEFT JOIN PageLikes l ON l.page_id = p.page_id
WHERE l.page_id IS NULL;
```

Explanation: Returns pages with no matching like records; adapt table names to the source schema.

7.6 App click-through rate (illustrative schema)

```
SELECT app_id,
       ROUND(100.0 * SUM(CASE WHEN event_type = 'click' THEN 1 ELSE 0 END)
       / NULLIF(SUM(CASE WHEN event_type = 'impression' THEN 1 ELSE 0 END), 0), 2) AS
ctr_pct
FROM Events
GROUP BY app_id;
```

Explanation: Calculates clicks as a percentage of impressions and avoids division by zero.

8. Important SQL Patterns Learned

- **Filtering: WHERE**
- **Sorting: ORDER BY**
- **Aggregation: COUNT, SUM, AVG, MIN, MAX**
- **Grouping: GROUP BY + HAVING**
- **Combining tables: INNER/LEFT/RIGHT/FULL/SELF JOIN**
- **Pattern matching: LIKE and wildcards**
- **Set operations: UNION**
- **Subqueries and CTEs**
- **Window functions such as DENSE_RANK()**
- **Self-joins for manager/employee comparisons**
- **NULL-safe conditions using IS NULL / IS NOT NULL**

9. SQL Best Practices

- Select only the columns required by the problem.
- Use meaningful aliases to make queries easier to read.
- Write explicit JOIN conditions and check for duplicate rows after joins.
- Use WHERE to filter individual rows and HAVING to filter aggregate groups.
- Handle NULL values with IS NULL or IS NOT NULL.
- Use common table expressions (CTEs) to clarify multi-step queries.
- Use window functions for ranking and calculations across related rows.
- Before UPDATE or DELETE, run a SELECT with the same WHERE clause to verify the affected records.
- Check SQL dialect differences for date arithmetic, string functions, and row-limiting syntax.

10. Conclusion

This project provides a structured path from fundamental SQL commands to advanced interview-style analytical problems. The roadmap covers the SQL operations commonly used for data retrieval, transformation, aggregation, and relational analysis. Practising each query against the correct platform schema, validating the output, and documenting the reasoning will strengthen practical SQL problem-solving skills and make the work easier to present in a professional portfolio.